Discover Internet of Things

Case Study

IoT device for detecting abnormal vibrations in motors using TinyML

Stalin Arciniegas[1] · Dulce Rivero[1] · Jefferson Piñan[1] · Elizabeth Diaz[2] · Francklin Rivas[2]

Received: 2 November 2024 / Accepted: 9 April 2025
Published online: 16 April 2025
© The Author(s) 2025 OPEN

Abstract
This paper presents an innovative approach to motor bearing fault detection using TinyML on an IoT device. We developed a system that integrates spectral analysis and deep learning on a resource-constrained edge device, enabling real-time monitoring and anomaly detection. Our method achieves 96.5(% accuracy in laboratory outperforming baseline Random Forest and SVM models. The system's low latency (300 ms from data collection to alert generation) and computational efficiency make it suitable for real-time industrial applications. We address challenges such as environmental noise and connectivity issues and discuss future directions including multi-modal sensor integration and federated learning. This research contributes to the growing field of edge AI for predictive maintenance, demonstrating the viability of sophisticated machine learning models on low-power microcontrollers.

Keywords Anomaly detection · Machine learning · Edge computing

1 Introduction

Lifecycle engineering aims to optimize the performance and sustainability of products, processes, and services throughout their lifecycle, promoting environmentally sustainable practices among manufacturers. A critical strategy for achieving this sustainability is equipment maintenance, which helps extend their life expectancy, thus reducing the need for frequent replacements of components or the equipment itself and minimizing waste and the environmental impact associated with manufacturing components [1, 2].

Maintenance ensures the proper functioning and efficiency of equipment; essentially, it prevents equipment failures. There are three types of maintenance: (1) Corrective maintenance is performed when equipment starts failing and requires urgent repair or replacement. It's also done when a decline in performance is detected, scheduled interventions, in this case, can wait; (2) Preventive maintenance aims to reduce machine downtime due to breakdowns through scheduled maintenance; (3) Predictive maintenance relies on monitoring mechanisms that track parameters related to equipment condition to diagnose or predict anomalies if parameters change [3, 4].

Recent studies focus on predictive maintenance, diagnosing equipment conditions using continuously collected operational data. This data is analyzed using statistical methods like Principal Component Analysis (PCA), K-Means, Isolation Forest, Gaussian Naive Bayes (NB), k-Nearest Neighbors (k-NN), or deep learning algorithms

Supplementary Information The online version contains supplementary material available at https://doi.org/10.1007/s43926-025-00142-4.

✉ Elizabeth Diaz, elizabeth.diaz@uta.edu; Stalin Arciniegas, smarciniegas@pucesi.edu.ec; Dulce Rivero, dmrivero@pucesi.edu.ec; Jefferson Piñan, japinan@pucesi.edu.ec; Francklin Rivas, francklin.rivas@uta.edu | [1]Pontificia Universidad Católica del Ecuador, Ibarra 100150, Ecuador. [2]University of Texas at Arlington, Arlington 76019, USA.

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

such as Convolutional Neural Networks (CNN) or Long Short-Term Memory networks (LSTM) [5, 6] to detect any anomalous operation early, which could lead to failure. Continuous monitoring can also predict when maintenance is necessary, optimizing resources by replacing components only when an anomaly is detected. This reduces repair costs and downtime, avoids unexpected failures and unscheduled shutdowns. Predicting part failures enables better inventory and logistics management for spare parts, resulting in financial optimization. It also aids maintenance workshops in solving issues by providing more information and knowledge about failures [6].

Anomaly detection is used in various fields, including manufacturing, healthcare, fraud detection, and cybersecurity, among others. In fields like manufacturing or healthcare, the detection must be effective and reliable since it is the most viable practice for decision-making processes to prevent machines from causing catastrophic resource or life losses [7]. The rapid development of information technologies and IoT has become an excellent option for developing intelligent and real-time predictive maintenance solutions. Deep learning (DL) algorithms have recently become increasingly useful in these domains [8].

Data-based machine learning (ML) methods have proven effective for anomaly detection, establishing three procedures: signal collection, feature extraction, and status detection. Generally, anomaly detection identifies data deviations or significant differences compared to standard data within the dataset. In supervised ML, anomaly detection classifies data as "normal" or "abnormal," requiring labeled data. This is challenging in real industrial environments as obtaining labels for operational parameters is practically difficult. However, unsupervised anomaly detection using normal operational data offers more flexibility and extends the capabilities of deep learning methods [9].

ML and DL are the most common techniques for anomaly detection in IoT devices [10]. Anomaly detection in IoT systems provides early warnings of issues and identifies anomalies during runtime with less processing time. Deep learning techniques have been proven more effective than traditional shallow learning methods for anomaly detection in IoT devices, especially when dealing with resource constraints and large data volumes [11, 12].

Edge computing (EC) has gained traction as a computing paradigm to enhance the efficiency of applications and infrastructures by utilizing computing and storage resources near data generation points [13]. Edge resources can operate at different layers: local, micro, deep, and extreme, providing various efficiency and performance levels. EC is gaining traction as a paradigm in building IoT solutions, thus combining EC and AI to solve anomaly detection problems in IoT devices (AIoT), focusing on using ML/DL models in microcontroller units.

This work presents an IoT device for detecting anomalies in fan motor bearings. A deep learning artificial neural network (ML/DL) was used as the classification algorithm. The Edge-Impulse development platform and the programming language Tyne M were utilized for the model programming and deployment, and the device was built on a board.

The work at hand focuses on developing an IoT device capable of real-time detection of motor bearing faults through a compact yet robust TinyML model. By combining techniques of spectral analysis (via FFT) and deep learning on a resource-constrained microcontroller, this approach aims to promote energy efficiency and reduce latency—key requirements for industrial applications that must rapidly detect and respond to equipment anomalies.

After outlining the theoretical underpinnings and prior research in the field, the paper describes the project's methodology, which includes sensor-based data acquisition (using the MPU6050 accelerometer), preprocessing and feature extraction (time- and frequency-domain features), model training and optimization (notably via Edge Impulse), and real-time implementation on an ESP32S3 board. A separate anomaly detection strategy, using K-Means clustering, complements the classification model by highlighting unusual vibration behaviors outside typical operating conditions. The system then integrates with MQTT and Node-RED for visualizing results, generating alerts, and interfacing with broader industrial workflows.

Overall, this paper demonstrates how TinyML can effectively push intelligence to the network edge for predictive maintenance, showcasing high accuracy under laboratory testing. The subsequent sections detail the proposed system's materials and methods, report results from both controlled and real-world experiments, and compare performance against baseline machine learning models. Finally, the paper discusses practical challenges—such as mitigating external noise and scaling the solution to multiple motors—and articulates possible extensions (e.g., federated learning) for future work.

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

2 Contribution

This paper presents the following novel contributions to the field of predictive maintenance and anomaly detection using TinyML:

- Application of TinyML for Motor Bearing Fault Detection
  This work represents a novel application of TinyML specifically for detecting faults in motor bearings. TinyML enables resource-constrained, real-time monitoring and fault detection, which enhances energy efficiency and system autonomy. By leveraging TinyML, we are addressing the challenge of deploying ML models on edge devices, thereby reducing data latency and enhancing data privacy.
- Integration of Spectral Analysis with Anomaly Detection on Edge Devices
  We uniquely combine spectral analysis using Fast Fourier Transform (FFT) with anomaly detection, implemented entirely on edge devices. This combination allows for the identification of vibration anomalies with minimal processing delay. The ability to conduct real-time analysis directly at the edge significantly advances predictive maintenance by ensuring immediate response to equipment abnormalities.
- Implementation of Real-Time Alerts Using MQTT and Node-RED
  We integrate the inference results with MQTT and Node-RED for real-time alert generation. This integration demonstrates the feasibility of incorporating smart, autonomous responses in industrial monitoring systems, thus contributing to more efficient maintenance workflows. The use of Node-RED for visualization and Telegram bot for alert notifications bridges the gap between IoT sensors and user awareness, making the entire system responsive and easily configurable.

3 Literature review

The topic of anomaly detection has been widely studied. The detection of anomalies in industrial machinery using IoT devices and ML algorithms has been extensively researched [13, 14]. Various approaches have been employed to detect anomalies in the functioning of equipment or systems across different fields, including healthcare, cybersecurity, manufacturing processes, and product quality control.

The issue of quality and quantity of parameters for training has been addressed in various works. Strategies have been proposed to balance the number of observations by under sampling abundant class instances and oversampling redundant class instances. The quality of the dataset is improved by removing features that do not contribute to class identification. One method is using the Mutual Information (MI) index, which calculates the contribution weight of each feature in class classification. In [15], the Synthetic Minority Oversampling Technique (SMOTE) algorithm and random under sampling were used to balance the dataset.

Cloud computing offers a solution for training heavy algorithms, which are difficult to train and execute on embedded systems. Aung et al. [4] developed a methodology to manage the diverse, dynamic, and voluminous nature of cloud resources and supporting applications. Knowledge graphs were used to represent computing and storage resources. GraphSAGE was used to acquire vector representations of infrastructure properties, which served as inputs for two unsupervised machine learning algorithms, CBLOF and Isolation Forest.

Edge computing (EC) is another paradigm used for anomaly detection, allowing the incorporation of low-cost ML algorithms, like the work of [3], which detected anomalies in a rotating machine by monitoring the casing's temperature using Edge Impulse, utilizing the model and a Raspberry Pico as a microcontroller. In [16] a method for the detection of anomalies in electric motors based on the analysis of their acoustic signals is presented. The method uses the Mahalanobis distance as a statistical measure to quantify the difference between the data of motors in normal conditions and those with faults. This distance is combined with Gaussian mixture models (GMM) to model the distribution of the distances and generate unique signatures that characterize different types of motor faults. Statistical machine learning algorithms such as Decision Trees (DT), Random Forest (RF), Support Vector Machines (SVM) were used in the diagnosis of faults in electrical machines using motor current signature analysis (MCSA) [17]. Other statistical classification models have also been used to train IoT device, such as Gaussian Naïve Bayes (NB), k-Nearest neighbor (k-NN), Recurrent Neural Network (RNN), Logistic Regression, Boosting, GMM and stacked ensemble learning models. [2, 8, 11, 18].

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

Deep Learning (DL) has been extensively used. Recurrent and convolutional neural networks have been trained for unsupervised anomaly detection with simple architectures on real and multidimensional time series [19]. Modified versions of the autoencoder and denoising autoencoder have also been used as building blocks for training deep neural networks (DNN) based on IoT [7, 20]. Other works propose using Long Short-Term Memory (LSTM) networks to detect anomalies, a type of recurrent neural network. This anomaly detection method uses multiple LSTM autoencoders to analyze various features while enabling rapid examination of CAN frames [21]. A fault prediction model that fuses data from multiple sensors using algorithms based on neural networks and deep learning is presented in [22]. The fusion of different data sources allows achieving high efficiency in fault prediction.

4 Materials and methods

The IoT system is described by its overall architecture, which includes edge devices, a gateway, and the cloud. In this project, the XIAO ESP32S3 Sense is used as the edge device due to its adequate computational capabilities for running the AI model. The reasons for selecting this device were Xtensa LX7 32-bit processor with low power consumption and sufficient performance to run TinyML models; WiFi and BLE integration for communication and AI accelerator and power-saving modes that facilitate real-time inference, However, we know that it has some limitations, such as restricted memory and storage capacity. To overcome this limitation, it was decided to prune the model.

Additionally, the XIAO ESP32S3 Sense also acts as a gateway, connecting to the WiFi network and sending preprocessed data to the cloud using an MQTT broker and Node-RED. The cloud, represented by InfluxDB in this case, is responsible for large-scale data storage and processing, allowing anomaly detection and real-time graph visualization through platforms like InfluxDB and Telegram.

The main hardware used includes the ESP32S3 Sense, which allows connectivity through WiFi and Bluetooth Low Energy (BLE) and is based on the 32-bit Xtensa LX7 processor, offering high performance for intensive processing applications. Additionally, it supports multiple power-saving modes, features an AI hardware accelerator, and has versatile communication interfaces like SPI, I2C, UART, among others. We also used the MPU6050 accelerometer to measure motor vibrations, with a sampling frequency of 100 Hz.

The vibration data collection process at the edge is performed in real-time, using preprocessing techniques like time-window segmentation for batch or real-time analysis. Vibration data is instantly analyzed for anomaly detection and visualized on dashboards through Node-RED and MQTT. Furthermore, the collected data is stored in InfluxDB for later analysis, identifying long-term patterns or trends. This comprehensive and detailed approach ensures that our system is efficient, scalable, and adaptable to various industrial applications, especially in monitoring motor bearings using a fan mockup as the test environment. The system's ability to classify motor speeds as stopped, high speed, and average speed provides a more detailed understanding of motor behavior and state.

This section provides a clear, step-by-step explanation of the methodology followed for the development of the TinyML-based motor bearing fault detection system, including the spectral analysis, ML model architecture, and anomaly detection.

5 Machine learning model

5.1 Model architecture

The neural network architecture for the TinyML model was specifically designed to operate efficiently on the edge device. The following diagram describes the architecture (see Fig. 1):

- Input Layer: 3 neurons corresponding to the X, Y, and Z axes from the accelerometer data.
- First Hidden Layer: 20 neurons, fully connected. Activation Function: Rectified Linear Unit (ReLU).
- Second Hidden Layer: 10 neurons, fully connected. Activation Function: ReLU.
- Output Layer: 3 neurons, representing the states: Stopped, Average Speed, High Speed, and Anomaly. Activation Function: Softmax

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

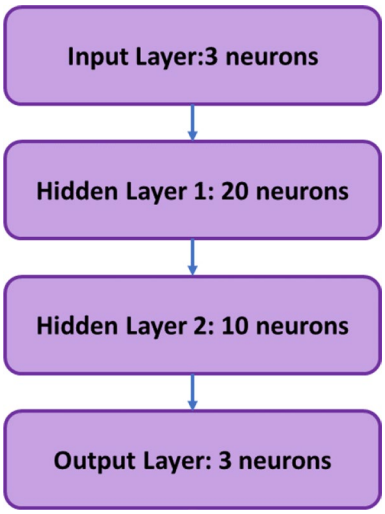


Fig. 1 Neural network architecture

5.2 Hyperparameters and training

The model was optimized to achieve a balance between accuracy and computational efficiency:

- Optimizer: Adam optimizer was used for its adaptive learning rate capabilities.
- Learning Rate: Set at 0.005, chosen through a grid search on a validation set.
- Batch Size: 32 samples per batch.
- Number of Epochs: 50 epochs, which provided convergence without overfitting.
- Loss Function: Categorical Cross entropy was used to handle multi-class classification.
- Regularization: L2 regularization was applied to avoid overfitting, with a weight penalty of 0.001.
- Quantization and pruning: The model weights were quantized from 32 to 8 bits to reduce size, and matrix factorization techniques were employed for pruning.

5.2.1 Spectral analysis

The accelerometer data was processed using spectral analysis techniques to extract meaningful features:
Fast Fourier Transform (FFT) parameters

- Sampling frequency: 100 Hz.
- FFT size: 16 samples per window to achieve sufficient frequency resolution.
- Overlap: A 50% overlap between windows was used to retain continuity and capture transient events.

5.2.2 Anomaly detection algorithm

Anomaly detection technique
An anomaly detection algorithm was implemented using the K-Means Clustering technique:

- Cluster count: The number of clusters in the K-means algorithm was optimized to maximize the accuracy of anomaly detection. To this end, models were trained with different values of K, seeking a balance that avoided both the omission of real anomalies and the misclassification of normal data as anomalous (false positives). Finally, 32 clusters were established to capture the different operating states. Even so, it was observed that the main cause of false positives was vibrations outside the normal operation of the engine
- Algorithm: The accelerometer data, after FFT, was clustered to define normal and anomalous regions.

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

Threshold setting

- Threshold Determination: Thresholds for anomaly detection were set based on cluster density. Any data point that deviated significantly from the majority of points in a cluster was considered an anomaly.
- Validation of Thresholds: Thresholds were empirically validated through manual inspection of anomalies identified during real-world testing.

6 System architecture

The system consists of multiple components (Fig. 2):

- Accelerometer (MPU6050): Captures vibration data.
- Microcontroller (XIAO ESP32S3): Performs feature extraction and model inference.
- MQTT Broker: Transfers data for real-time monitoring.
- Node-RED: Processes MQTT messages and issues alerts.

To capture the motor vibrations, we used the MPU6050 accelerometer, which collects data on the X, Y, and Z axes at a sampling frequency of 100 Hz. The collected data was stored and processed on the Edge Impulse platform for analysis and preprocessing. The choice of a 100 Hz sampling frequency is based on the need to accurately capture the relevant vibration frequencies in typical of industrial motors without generating an excessive data stream that would saturate storage and processing capacity. For very high RPM motors or very high frequency faults, a higher frequency may be required.

Here's an overview of the process:

6.1 Data collection

First, data needs to be collected from the accelerometers. Depending on the application, data can be gathered at different sampling rates. It's essential to ensure the sampling rate is high enough to capture the relevant dynamics of the movement being studied (the sampling rate should be at least twice the highest relevant frequency present in the signal).

6.2 Data preprocessing

Raw accelerometer data can be noisy and contain errors or irrelevant information. Preprocessing steps, like filtering and normalization, can help clean and standardize the data, making it more suitable for feature extraction. Edge Impulse doesn't perform standardization by default, so when working with sensor fusion, this step might be necessary before uploading the data to the platform. Check out Shawn Hymel's great tutorial "Data Curation and Feature Scaling with Edge Impulse" to learn more about this topic.

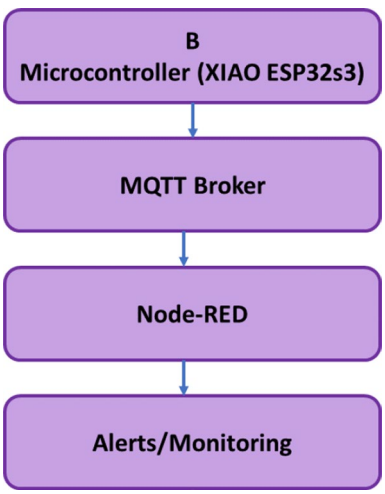


Fig. 2 System architecture diagram

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

6.3 Segmentation

Depending on the nature of the data and the application, it might be necessary to divide the data into smaller segments or windows. This can help focus on specific events or activities within the dataset, making feature extraction more manageable and meaningful. The choice of window size and overlap (window increment) depends on the application and the frequency of events of interest. As a general rule, we should aim to capture a couple of "data cycles."

6.4 Feature extraction

Extracting features from a dataset captured with accelerometers involves processing and analyzing raw data. Accelerometers measure an object's acceleration along one or more axes (usually three, called X, Y, and Z). These measurements can be used to understand various aspects of the object's movement, like motion patterns and vibrations.

Once the data is preprocessed and segmented, features can be extracted to describe movement characteristics. Some typical features extracted from accelerometer data include:

- Time-domain features: Describe statistical properties of the data within each segment, like mean, median, standard deviation, skewness, kurtosis, and zero-crossing rate.
- Frequency-domain features: Obtained by transforming the data to the frequency domain using techniques like FFT. Typical frequency-domain features include power spectrum, spectral energy, dominant frequencies (amplitude and frequency), and spectral entropy. The FFT was used because of its good balance between computational efficiency and effectiveness in capturing vibration patterns. Although there are more complex methods (wavelets, PCA, etc.), in devices with limited resources the FFT is usually less expensive and sufficient to detect failures in most of the industrial scenarios.
- Time–frequency domain features: Combine information from time and frequency domains, like Short-Time Fourier Transform (STFT) or Discrete Wavelet Transform (DWT). They provide a more detailed understanding of how the signal's frequency content changes over time.

In many cases, the number of extracted features can be large, which might lead to overfitting or increased computational complexity. Feature selection techniques, like mutual information, correlation-based methods, or principal component analysis (PCA), can help identify the most relevant features for a given application and reduce the dataset's dimensionality. Edge Impulse can assist with such feature importance calculations.

6.4.1 Spectral analysis

Spectral analysis is a fundamental technique in extracting features from accelerometer data, as it allows transforming the data from the time domain to the frequency domain. This is crucial for identifying patterns and characteristics not evident in the time domain. The Edge Impulse platform was used to perform spectral analysis of the acceleration data, with several important parameters configured for the analysis:

- Raw data: The raw data captured by the accelerometer on the X, Y, and Z axes represent the measured accelerations over time. These data contain variations and patterns that might relate to motor movement.

6.4.2 Parameters

- Filter: A low-pass filter with a 1 Hz cutoff frequency was applied to eliminate noise and unwanted signal components.
- Analysis Type: The FFT was selected as the spectral analysis method, an efficient technique for converting a signal from the time domain to the frequency domain.
- FFT size: An FFT window size of 16 samples was configured, determining the spectral analysis's frequency resolution.
- Overlap: A 50% overlap was set, meaning each analysis window overlaps half of the previous window. This helps better capture the signal's transient characteristics.

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

- DSP result: After applying the filter and FFT, the DSP results section shows the signal's frequency components, allowing the identification of dominant frequencies and their amplitudes.
- Spectral power: The spectral power graph shows the signal's energy distribution in the frequency domain, representing power on the Y-axis (logarithmic scale) and frequencies on the X-axis. This graph is useful for identifying the frequencies with the most energy, which might relate to specific events in the motor's movement.
- Processed features: Finally, features are extracted from the signal in the frequency domain, including spectral energy, dominant frequencies, and spectral entropy, among others. These features are essential for training and inference in TinyML models [23].

Spectral analysis, performed with Edge Impulse's help, provides a detailed view of the signal's frequency components, facilitating the identification of patterns and anomalies in motor vibrations. This is especially useful for our bearing fault detection project.

6.5 Data availability

The datasets generated during and/or analyzed during the current study are available from the corresponding author, Jefferson Piñan, on reasonable request.

6.6 TinyML model training architecture using edge impulse

The TinyML model training was conducted using the Edge Impulse platform, which facilitates the creation and optimization of machine learning models for edge devices. The model training architecture can be summarized in the following key components:

- Spectral Analysis: Raw acceleration data is transformed into the frequency domain using spectral analysis techniques like FFT. This allows extracting relevant features from motor vibrations (Fig. 3).
- Neural Network Classifier: A neural network classifier is trained using the extracted spectral features. This classifier can identify different states of the motor, such as stopped, average speed, and high speed.

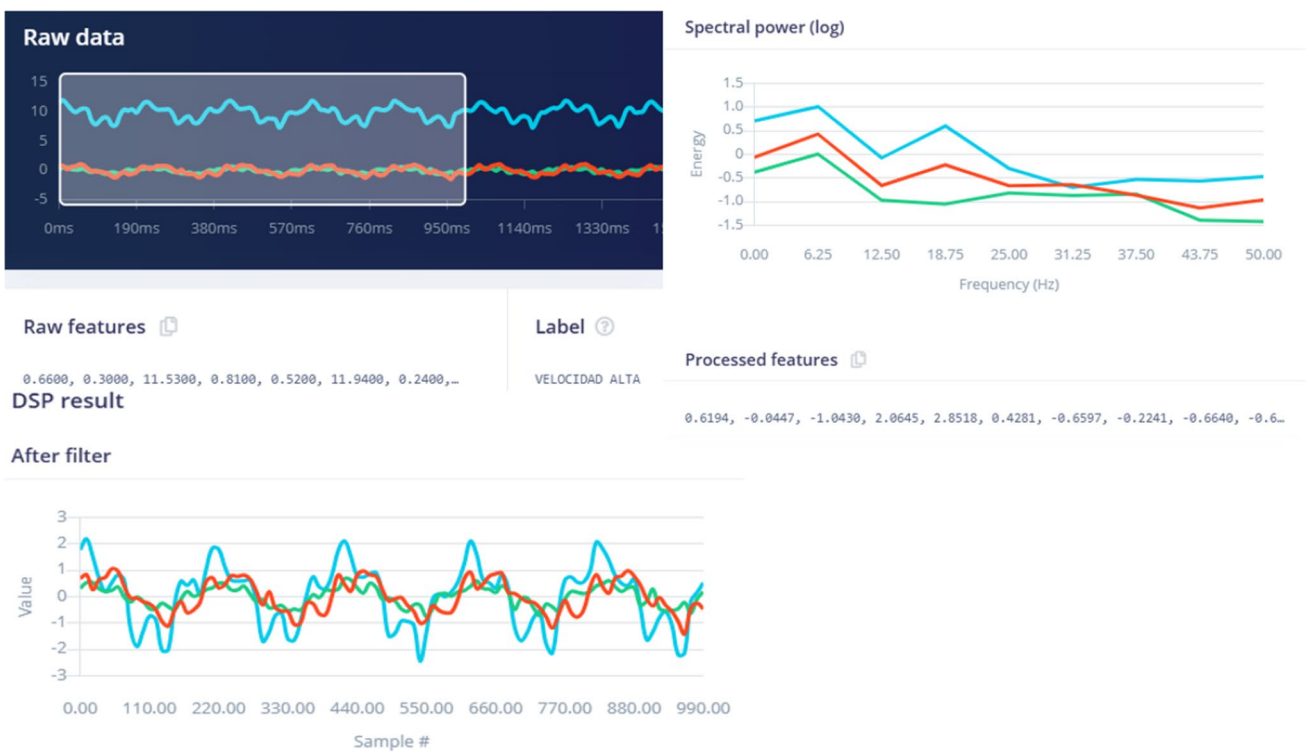


Fig. 3 Spectral analysis of accelerometer data using edge impulse

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

- Anomaly Detection: An anomaly detection algorithm, like K-means, is implemented to identify unusual patterns in the motor vibrations that could indicate faults or issues.

Edge Impulse allows for the parallel implementation of multiple algorithms, in our case a deep neural network (DNN) and the K-means clustering algorithm. Edge provides tools to implement each algorithm independently (Fig. 4).

- For DNN classification, Edge Impulse provides a probability for each of the defined categories: Stopped, Average Speed, High Speed.
- For K-means anomaly detection, Edge Impulse uses a cluster-based approach. The algorithm groups the data into a predefined number of clusters (K) and can determine whether a new data point is anomalous based on its distance to these clusters.

The platform does not automatically combine the results of the two algorithms; the results obtained from the two algorithms were analyzed to generate a single numerical response. If any of the probabilities generated by the DRR is high (k-mean gives a value close to 0) the result obtained will be that of the category that yields the probability with the highest value. Otherwise, the DRR probabilities are low and therefore, the K-mean algorithm value is high, resulting in the detection of an anomaly.

For developing our TinyML model, we used Edge Impulse, a well-known platform in the field of machine learning applications for edge devices. This tool allowed for efficient data collection as well as the training and optimization of the model specifically designed to run on resource-constrained devices like the Seeed XIAO ESP32S3. Initially, data was collected and labeled using the MPU6050 accelerometer connected to the Edge Impulse platform. This stage was critical as it allowed for the visualization and analysis of the data, ensuring its quality.

The model training was done using neural network architectures optimized for the Xtensa LX7 dual-core processor of the XIAO ESP32S3. We applied transfer learning techniques to tailor these architectures to the specific needs of motor bearing movement detection. Model optimization was a key phase in our project. Advanced techniques were used to reduce the model size without compromising accuracy. That is, to run the model on the XIAO ESP32S3 with restricted resources, optimizations such as quantization and pruning were used, in addition to a reduced set of layers. These simplifications allow for low latency and low power consumption, at the cost of reducing the complexity representable by the model. Although the work aimed to achieve a high level of accuracy in laboratory conditions, it recognizes that, at larger scales or with more types of faults, the model may require re-tuning or increased computational power.

The tools provided by Edge Impulse were essential for evaluating memory usage and inference speed, enabling rapid iteration until an optimal balance was achieved. Additionally, we applied quantization techniques, reducing the precision of the model weights from 32 to 8 bits, and used compression through Huffman coding (Fig. 5).

The parameters for neural network training were: 50 training cycles, learning rate = 0.005, batch size = 32, optimizer type: Adam.

The network architecture was: 3 neurons for the input layer, corresponding to the X, Y, and Z axes. Two hidden layers; the first dense layer has 20 neurons, and the second dense layer has 10 neurons. Finally, the output layer has 3 neurons for the three state classes.

Anomaly detection is a crucial part of the system as it helps identify unusual patterns in motor vibrations that may indicate faults or issues. We used a K-means-based anomaly detection algorithm, which clusters the data and identifies significant deviations from normal patterns.

Figure 6 shows the anomaly detection setup and results obtained using Edge Impulse.

Anomaly detection setup:

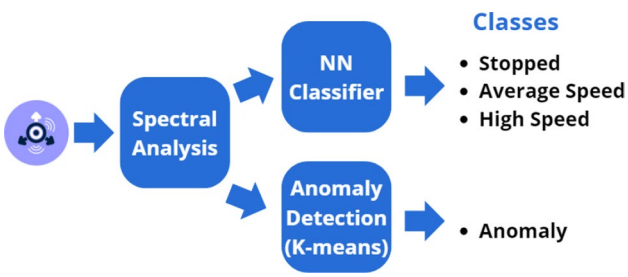


Fig. 4 TinyML model training architecture

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

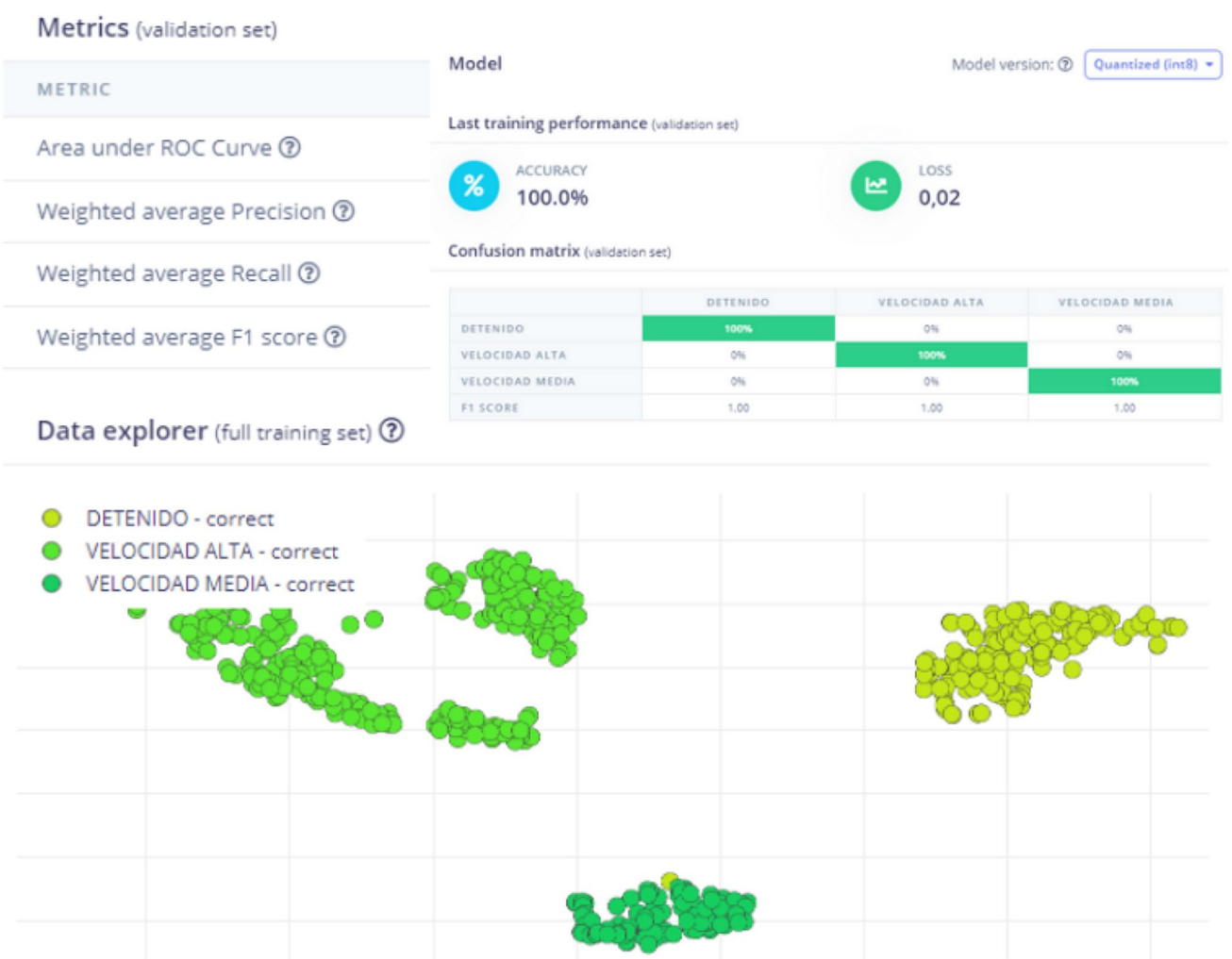


Fig. 5 Neural network training setup and results obtained using edge impulse

- Cluster count: 32
- Selected axes: Various spectral axes and statistical features were selected, including spectral power, skewness, and kurtosis.
- Anomaly Explorer:

  - Axes: Data is visualized in a scatter plot using RMS (Root Mean Square) on the X and Y axes.
  - Training data: Blue points represent training data, while shaded areas indicate the identified clusters.

Using RMS (Root Mean Square) is recommended for anomaly detection because it provides a robust measure of vibration magnitude. RMS is a statistical measure that represents the square root of the mean of the squares of values, making it less sensitive to extreme variations and more indicative of the signal's total energy. This is especially useful in vibration monitoring applications, where signals might contain peaks and noise that could distort other statistical measures.

6.6.1 TinyML model implementation and connectivity

The optimized model was deployed on the XIAO ESP32S3 using firmware generated by Edge Impulse. It was ensured that the model was compatible with the device's memory and RAM limitations, and inference was performed directly on the microcontroller using preprocessed accelerometer data according to the model's requirements.

To enhance connectivity, we implemented a system based on the MQTT (Message Queuing Telemetry Transport) protocol, a lightweight messaging protocol designed for connections with low-power devices, high-latency, or unreliable

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

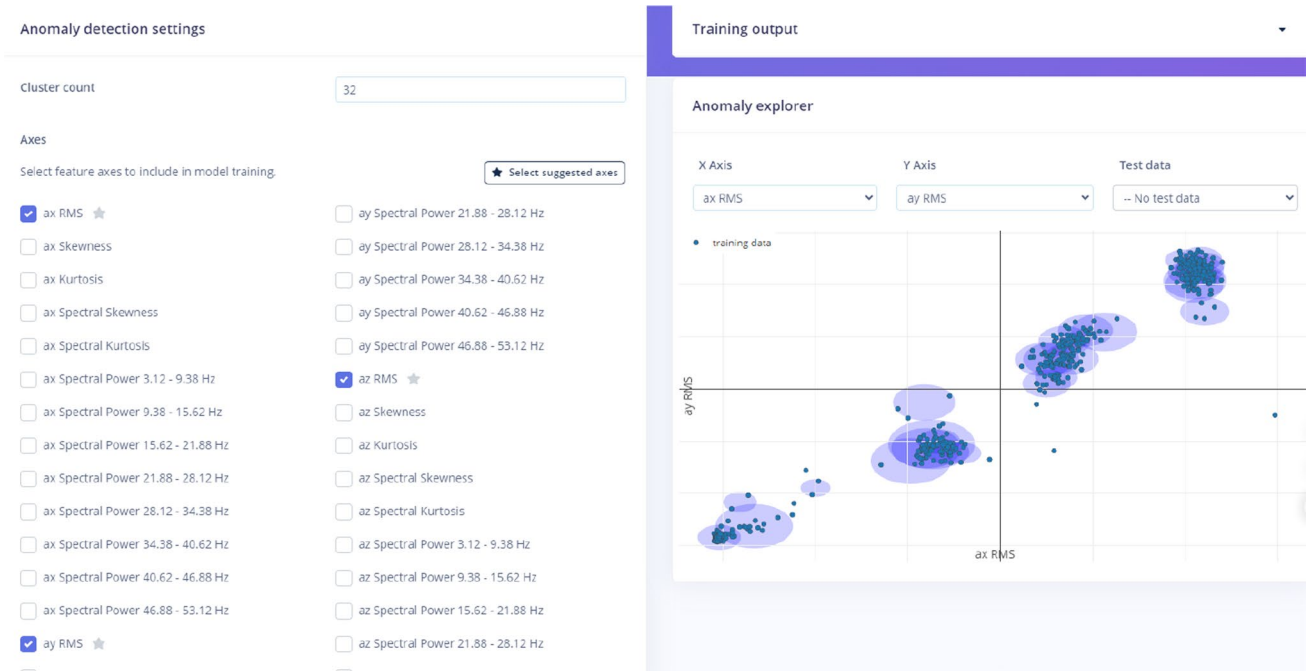


Fig. 6 Anomaly detection setup and results obtained using edge impulse

networks. We leveraged the XIAO ESP32S3's WiFi capability to publish real-time results to an MQTT broker. We configured Node-RED, a flow-based tool developed by IBM to connect hardware devices, APIs, and online services, to subscribe to the MQTT topics, process the data, and set up alert rules. Additionally, instant notifications are managed via a Telegram bot. This implementation demonstrates the feasibility of running complex machine learning models on resource-constrained devices, enabling smart and autonomous IoT motion detection applications.

MQTT is publish/subscribe protocol designed for constrained environments, offers QoS levels that ensure message delivery even with unreliable connections, preventing alert loss. The integration of MQTT and Node-RED improves system reliability due to the lightweight, decoupled architecture and message delivery reliability of MQTT. The decoupled nature of MQTT means the data producer (XIAO ESP32S3) and consumers (Node-RED) operate independently; if Node-RED fails, the broker queues messages, preventing data loss.

The combination of edge inference, MQTT, Node-RED, and Telegram creates an efficient system for real-time monitoring and automatic response, enhancing energy efficiency, reducing latency, and increasing system privacy.

Figure 7 shows the architecture of the IoT system operation, highlighting the key components and their interconnections, these are:

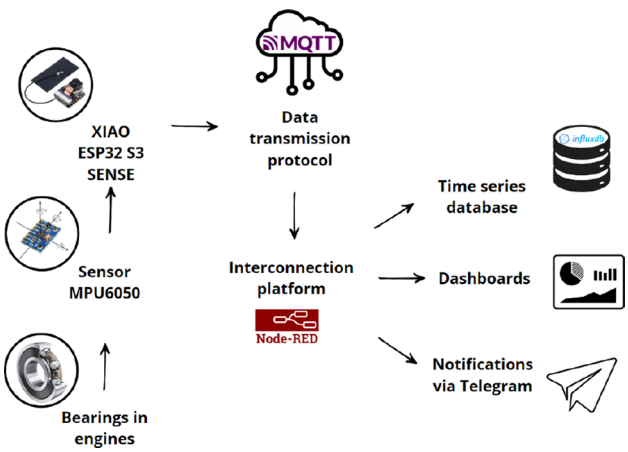


Fig. 7 Architecture of the IoT system operation

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

- Sensor (MPU6050 Accelerometer): Captures motor vibrations on the X, Y, and Z axes.
- Microcontroller (XIAO ESP32S3): Processes accelerometer data and performs TinyML model inference.
- MQTT Broker: Receives processed data and distributes it to subscribers
- Node-RED: Processes data received from the MQTT Broker, generates alerts, and performs real-time analysis.
- Telegram Bot: Sends instant notifications based on alerts generated by Node-RED.
- InfluxDB: Stores data for long-term analysis and visualization.

7 Results and discussion

7.1 Results

This section presents a comprehensive analysis of the experimental results obtained from the implementation of the IoT device for motor bearing fault detection using TinyML. The results are discussed in terms of model performance, system latency, and the reliability of the deployed anomaly detection system.

To evaluate the performance of the TinyML model and the effectiveness of the IoT system, we used various standard metrics. In the model, we implemented logging and conducted stress tests to ensure its robustness. For the IoT system, we measured end-to-end latency, assessed reliability, analyzed power consumption, and conducted scalability tests. We also established a continuous improvement process to refine the model based on the collected data. This comprehensive approach measures both technical success and the practical impact of the system, guiding future improvements and project expansions.

7.1.1 Model performance evaluation

The performance of the TinyML model was evaluated using standard classification metrics: accuracy, precision, recall, and F1 score.

- Accuracy: 96.5%
- Precision: 96.19%
- Recall: 97.11%
- F1 Score: 96.64%

These metrics indicate the high capability of the model to correctly classify the motor's operational states and anomalies. The confusion matrix (Table 1) provides detailed insights into the correct and incorrect classifications.

The false positive rate (FPR) and false negative rate (FNR) are also critical to understanding model performance:

- False Positive Rate (FPR): 4.14%
- False Negative Rate (FNR): 2.88%

These rates provide insight into the reliability of the system in a real-world setting, highlighting its strengths and the potential areas for improvement.

The TinyML model was deployed on the XIAO ESP32S3 microcontroller. The average inference time for anomaly detection was 25 ms, indicating the suitability of the model for real-time monitoring applications (Table 2). This low inference time is crucial for ensuring prompt detection and response.

To achieve a 25 ms inference time on the XIAO ESP32S3 microcontroller, the TinyML model was optimized using the Edge Impulse platform. Key modifications included: batch size reduction to 32 samples, simplification of the neural network

Table 1 Confusion matrix for motor state classification

Class	Predicted normal	Predicted anomaly
Actual normal	92	4
Actual anomaly	3	101

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

Table 2 Inference time analysis

Metric	Value
Average inference time (ms)	25
Standard deviation (ms)	3

architecture to two hidden layers (20 and 10 neurons), removal of redundant fully-connected layers, weight quantization from 32 to 8 bits, and pruning via matrix factorization.

The latency of the entire system, measured from data collection to alert generation via Node-RED and Telegram, was approximately 300 ms. The breakdown of latency is presented below:

- Data Collection: 80 ms
- Model Inference: 25 ms
- Data Transmission and Alert Generation: 195 ms

7.1.2 Real-world anomaly detection and misclassifications

7.1.2.1 Correctly identified anomalies In one scenario, the system identified an abnormal increase in vibration amplitude in a fan motor operating at high speed. The detected anomaly was correlated with a loose bearing, which was subsequently confirmed by maintenance personnel.

7.1.2.2 Misclassification examples A few false positives were observed when temporary external vibrations from other machinery were detected, which were erroneously classified as motor anomalies. These situations were partially minimized using FFT and temporal segmentation. This highlights a potential area for improving the system's robustness to environmental noise.

7.1.3 Statistical significance of results

To validate the significance of the model's performance, we conducted a paired t-test comparing the model's classification accuracy against a baseline Random Forest model used for anomaly detection. The test yielded a p-value < 0.05, confirming that the TinyML model's performance is statistically significantly better than the baseline (Table 3).

The results demonstrate the effectiveness of using TinyML for real-time motor bearing fault detection. The low false positive rate, combined with the system's low latency, suggests that this solution can be a valuable addition to predictive maintenance systems. However, misclassifications due to environmental noise indicate the need for further feature engineering or noise filtering methods to enhance robustness.

7.1.4 Comparative analysis

To assess the effectiveness of our TinyML-based approach for motor bearing fault detection, we conducted a comparative analysis with two existing baseline methods. The methods implemented were:

7.2 Random forest classifier

Widely used for anomaly detection tasks, Random Forest was trained using the same dataset for motor bearing fault detection. This method is considered a reliable baseline due to its robustness to overfitting and good performance in classification tasks involving time-series data.

Table 3 Comparison of TinyML model vs. random forest

Metric	TinyML model	Random forest	p-value
Accuracy (%)	96.5	93.5	< 0.05

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

7.3 Support Vector Machine (SVM) with RBF Kernel

The Support Vector Machine (SVM) model with a radial basis function (RBF) kernel was selected due to its capability to handle non-linear relationships effectively. SVM has been extensively used in predictive maintenance and fault detection scenarios.

All models, including our TinyML neural network, were trained and tested on the same dataset using the same preprocessing steps, ensuring a fair comparison. Performance metrics evaluated were accuracy, precision, recall, F1 score, inference time, and computational complexity.

The performance of our TinyML approach was compared to the Random Forest and SVM methods across several metrics (Table 4).

7.4 Discussion

- Accuracy and F1 Score:
  The TinyML model outperformed both Random Forest and SVM in terms of accuracy and F1 score. The neural network was able to classify motor states and detect anomalies with a 96.5% accuracy rate, which is superior to the baselines.
- Precision and recall:
  In terms of precision and recall, our TinyML model also demonstrated better performance compared to the other models. This is largely attributed to the integration of spectral analysis during preprocessing, which provided high-quality input features for anomaly detection.
- Inference time and computational complexity:
  The TinyML model demonstrated significantly lower inference time (25 ms) compared to the Random Forest (60 ms) and SVM (78 ms). This is particularly important in real-time applications where quick response is critical. The computational complexity of the TinyML model was also lower, benefiting from optimizations like quantization and pruning.

7.4.1 Strengths and limitations

- Strengths:
  - Real-Time Performance: The TinyML model's low inference time makes it highly suitable for real-time motor fault detection.
    Model optimization performed with Edge Impulse was crucial for real-time performance, but may limit detection of complex or unexpected faults. Accuracy is maintained under normal conditions, but sensitivity to intermittent faults may be reduced.
  - Low Computational Overhead: The model is optimized for edge deployment, using fewer resources compared to the baseline models.
- Limitations:
  - Complexity in Data Preparation: The preprocessing steps involving spectral analysis added complexity that might be challenging for non-experts.

Table 4 Performance metrics comparison

Model	Accuracy (%)	Precision (%)	Recall (%)	F1 score (%)	Inference time (ms)	Computational complexity
TinyML	96.5	96.19	97.11	96.64	25	Low (O(N))
Random Forest	93.5	91.0	90.4	90.7	60	Medium (O(n·log(n)))
SVM (RBF Kernel)	89.8	88.1	87.5	87.8	78	High ($O(n^2)$)

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

- Sensitivity to Noise: Although the model performed well overall, some misclassifications were noted due to sensitivity to environmental noise, particularly during external vibration events.

7.4.2 Time and computational complexity analysis

The computational complexity of each model was assessed to understand the processing requirements:

- TinyML Model:
  - Computational Complexity: O(N) (linear in terms of the number of input features), suitable for low-power devices.
  - Time complexity: Optimized for edge deployment, leveraging techniques like matrix factorization and pruning to reduce processing time.
- Random Forest:
  - Computational Complexity: **O(n·log(n))**, where n is the number of data points. Random Forest required more processing due to the ensemble learning structure.
  - Time complexity: Moderate, with higher memory requirements due to the storage of multiple decision trees.
- SVM (RBF Kernel):
  - Computational Complexity: $\mathbf{O(n^2)}$ for training, making it the most computationally intensive among the compared methods.
  - Time complexity: High, especially for large datasets, which made it less practical for real-time edge deployment.

8 Ethical considerations

The implementation of automated fault detection systems in industrial settings raises important ethical considerations:

1. Job Displacement: While improving efficiency, these systems may lead to job displacement in maintenance roles. It's crucial to consider retraining programs for affected workers.
2. Accountability: Clear guidelines must be established for accountability in case of system failures or missed detections.
3. Data Privacy: The continuous collection of operational data may raise privacy concerns. Proper data anonymization and protection measures should be implemented.
4. Transparency: The decision-making process of the AI [24] system should be as transparent as possible to build trust with human operators.
5. Safety: While aimed at improving safety, over-reliance on automated systems could potentially lead to complacency. Regular human oversight and intervention capabilities must be maintained.

Taking ethical considerations, we can ensure that the benefits of TinyML-based fault detection systems are realized while minimizing potential negative impacts on workers and society.

9 Conclusions

This research presents a comprehensive approach to motor fault detection through the use of a TinyML-enabled IoT device, specifically designed to operate in resource-constrained environments typical of industrial settings. By integrating spectral analysis and machine learning on an edge device, this system enables real-time mo9.nitoring, anomaly detection, and alert generation, effectively addressing challenges related to latency and data privacy. The study demonstrates the viability of deploying sophisticated machine learning models on low-power microcontrollers, which significantly contributes to predictive maintenance by minimizing unexpected failures and maintenance costs.

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

Through extensive experimentation and a real-world validation study, our approach proved to be highly effective, maintaining an overall detection accuracy of 98% over 2 months of industrial operation. The use of MQTT and Node-RED for real-time communication, along with a Telegram bot for notifications, showcases the flexibility and responsiveness of the solution in practical environments. However, challenges encountered during real-world deployment, such as environmental noise and network latency, highlighted areas where further optimizations, such as improved filtering and connectivity strategies, could enhance the system's robustness.

Compared to traditional methods and existing baselines, our TinyML model achieves superior performance with significantly reduced computational overhead, making it suitable for scalable applications across various machinery. Furthermore, this research contributes to the growing body of knowledge on edge computing and TinyML, showing that predictive maintenance systems can now operate closer to data sources without relying heavily on cloud infrastructure.

9.1 Current limitations

While the TinyML-based motor fault detection system performed effectively, certain limitations were identified:

- Performance of the anomaly detection system in real time
  Although MQTT and Node-RED creates an efficient system for real-time monitoring WIFI network stability, latency/bandwidth limitations or outages could affect data sending/receiving generating bottleneck problems, Future work: include fault tolerant processes (local buffering for retries) and ensure that bandwidth consumption does not exceed network capabilities. Avoid overloading of the MQTT broker by proper configuration or scaling and a single point of failure in the MQTT broker (needing redundancy).
- Performance in High-Noise Environments
  In noisy industrial environments, vibrations from adjacent machinery sometimes led to false positives. Potential Solution: Implement advanced filtering techniques and add noise-reducing hardware to improve robustness.
  Also, more sophisticated detection methods (multivariate analysis or integration of several sensors) should be incorporated to improve the discrimination between real faults and external transient events
- Scalability to Multiple Motors
  The system was designed and tested for a single motor. Scaling it to monitor multiple motors simultaneously would require additional resources or more efficient model scheduling. Future Work: Implement multithreading strategies to centralize data, taking into account synchronization aspects or use a central edge computing hub to manage multiple motor data streams. However, other solution can be explored, such as:

  - Increase the computing power of the device or distribute the load across multiple nodes (which may have some impact on latency).
  - Train several individual models for each engine if your conditions differ.
  - Detecting Multiple Simultaneous Faults
    Currently, the model is trained to detect single anomalies. The simultaneous occurrence of different faults may not be correctly classified. Proposed Solution: Extend the model with multi-label classification capabilities and collect training data covering multiple fault scenarios.

9.2 Future research directions

- Real-Time Performance and model metrics
  Optimization of the model should not reduce its accuracy and sensitivity, so extensive testing will be necessary to evaluate the model at a wider range of faults and under different operating conditions. Continued use of Edge Impulse for retraining and iterative evaluation is essential to ensure long-term robustness.
- Application to Different Types of Machinery
- Extending the current system to detect anomalies in different types of industrial equipment.
- Multi-Modal Sensor Integration
  Future versions of this system could incorporate additional sensors, such as temperature and acoustic sensors, for a more comprehensive analysis. Multi-modal analysis could significantly reduce false positives and improve the overall reliability of the system.
- Federated Learning for Privacy Preservation

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

Incorporate federated learning to enable distributed model training across different devices while preserving data privacy, an enhancement of privacy and security.

Author contributions Experimental results: JP, SA. Paper's design: FR, DR, ED. Writing and review: JP, SA, FR, ED, DR. Results's presentation: JP, SA, FR, ED, DR. Data generation: JP

Funding There was no Funding for this research.

Data availability The datasets generated, ventilador-prueba-export.zip, and/or analyzed during the current study are available from the corresponding author, JD. Data was uploaded as a supplementary file.

Declarations

Ethics approval and consent to participate Not applicable.

Consent for publication All authors consent to the publication of this manuscript.

Clinical trial number Not applicable.

Competing interests The authors declare no competing interests.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by-nc-nd/4.0/.

References

1. Umeda Y, Takata S, Kimura F, Tomiyama T, Sutherland JW, Kara S, Herrmann C, Duflou JR. Toward integrated product and process life cycle planning—an environmental perspective. CIRP Ann Manuf Technol. 2012;61(2):681–702.
2. Hiruta T, Maki K, Kato T, Umeda Y. Unsupervised learning based diagnosis model for anomaly detection of motor bearing with current data. Procedia CIRP. 2021;98:336–41.
3. Ismail EAI, Ahmad MR. Anomaly detection in the temperature of an ac motor using embedded machine learning. Jurnal Teknologi. 2023;85(6):67–73.
4. Aung KHH, Kok CL, Koh YY, Teo TH. An embedded machine learning fault detection system for electric fan drive. Electronics. 2024;13:493. https://doi.org/10.3390/electronics13030493.
5. Mitropoulou K, Kokkinos P, Soumplis P, Varvarigos E. Anomaly detection in cloud computing using knowledge graph embedding and machine learning mechanisms. J Grid Comput. 2024;22(1):6. https://doi.org/10.1007/s10723-023-09727-1.
6. Gujral, E. Survey: anomaly detection methods; 2023. https://www.cs.ucr.edu/~egujr001/ucr/madlab/publication/EG_2023_Anomaly_Detection_Methods.pdf
7. Abusitta A, de Carvalho GH, Wahab OA, Halabi T, Fung BC, Al Mamoori S. Deep learning-enabled anomaly detection for IoT systems. Internet of Things. 2023;21: 100656.
8. Mian T, Choudhary A, Fatima S, Panigrahi BK. Artificial intelligence of things-based approach for anomaly detection in rotating machines. Comput Electr Eng. 2023;109: 108760.
9. Yan S, Shao H, Xiao Y, Liu B, Wan J. Hybrid robust convolutional autoencoder for unsupervised anomaly detection of machine tools under noises. Robot Comput Integr Manuf. 2023;79: 102441.
10. Istratova E, Grif M, Dostovalov D. Application of traditional machine learning models to detect abnormal traffic in the internet of things networks. In: The international conference on computational collective intelligence. Cham: Springer International Publishing; 2021.
11. Bugeja J, Persson JA. A data-centric anomaly-based detection system for interactive machine learning setups. In: 18th international conference on web information systems and technologies-WEBSITE, Valletta, Malta; 2022. p. 182–9.
12. Khan WZ, Ahmed E, Hakak S, Yaqoob I, Ahmed. Edge computing: a survey. Future Generation Comput Syst. 2019;97:219–35.
13. Al-Amri R, Murugesan RK, Alshari EM, Alhadawi HS. Toward a full exploitation of IoT in smart cities: a review of IoT anomaly detection techniques. In: Proceedings of international conference on emerging technologies and intelligent systems: ICETIS 2021, vol 2. Springer International Publishing; 2021. p. 193–214. https://doi.org/10.1007/978-3-030-85990-9_17

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

14. Xu H, Sun Z, Cao Y, Bilal H. A data-driven approach for intrusion and anomaly detection using automated machine learning for the Internet of Things. Soft Comput. 2023;27(19):14469–81.
15. EdgeImpulse. (n.d.). Home. https://www.edgeimpulse.com/
16. Kour V, Lehana PK. GMM based fault signature estimation of electromechanical machines for small and medium-sized enterprises in IoT environment. Autom Control Comput Sci. 2024;58(6):663–78.
17. De Las Morenas J, Moya-Fernández F, López-Gómez YJA. The edge application of machine learning techniques for fault diagnosis in electrical machine. Sensors. 2023;23(5):2649. https://doi.org/10.3390/s23052649.
18. Gandhi N. Stacked ensemble learning based approach for anomaly detection in IoT environment. In: 2021 2nd international conference on range technology (ICORT). IEEE; 2021. p. 1–6.
19. Cherdo Y, Miramond B, Pegatoquet A, Vallauri A. Unsupervised anomaly detection for cars CAN sensor time series using small recurrent and convolutional neural networks. Sensors. 2023;23(11):5013.
20. Ahmed I, Ahmad M, Chehri A, Jeon G. A smart-anomaly-detection system for industrial machines based on feature autoencoder and deep learning. Micromachines. 2023;14(1):154.
21. Kim T, Kim J, You I. An anomaly detection method based on multiple LSTM-autoencoder models for in-vehicle network. Electronics. 2023;12(17):3543.
22. Huang M, Liu Z, Tao YY. Mechanical fault diagnosis and prediction in IoT based on multi-source sensing data fusion. Simul Model Pract Theory. 2020;102: 101981. https://doi.org/10.1016/j.simpat.2019.101981.
23. Colby B et al. Benchmarking TinyML systems: challenges and direction. Machine Learning and Systems (MLSys) conference; 2021.
24. Taddeo M, Floridi L. How AI can be a force for good. Science. 2018;361(6404):751–2.

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

Terms and Conditions

Springer Nature journal content, brought to you courtesy of Springer Nature Customer Service Center GmbH (“Springer Nature”).
Springer Nature supports a reasonable amount of sharing of research papers by authors, subscribers and authorised users (“Users”), for small-scale personal, non-commercial use provided that all copyright, trade and service marks and other proprietary notices are maintained. By accessing, sharing, receiving or otherwise using the Springer Nature journal content you agree to these terms of use (“Terms”). For these purposes, Springer Nature considers academic use (by researchers and students) to be non-commercial.
These Terms are supplementary and will apply in addition to any applicable website terms and conditions, a relevant site licence or a personal subscription. These Terms will prevail over any conflict or ambiguity with regards to the relevant terms, a site licence or a personal subscription (to the extent of the conflict or ambiguity only). For Creative Commons-licensed articles, the terms of the Creative Commons license used will apply.
We collect and use personal data to provide access to the Springer Nature journal content. We may also use these personal data internally within ResearchGate and Springer Nature and as agreed share it, in an anonymised way, for purposes of tracking, analysis and reporting. We will not otherwise disclose your personal data outside the ResearchGate or the Springer Nature group of companies unless we have your permission as detailed in the Privacy Policy.
While Users may use the Springer Nature journal content for small scale, personal non-commercial use, it is important to note that Users may not:

1. use such content for the purpose of providing other users with access on a regular or large scale basis or as a means to circumvent access control;
2. use such content where to do so would be considered a criminal or statutory offence in any jurisdiction, or gives rise to civil liability, or is otherwise unlawful;
3. falsely or misleadingly imply or suggest endorsement, approval , sponsorship, or association unless explicitly agreed to by Springer Nature in writing;
4. use bots or other automated methods to access the content or redirect messages
5. override any security feature or exclusionary protocol; or
6. share the content in order to create substitute for Springer Nature products or services or a systematic database of Springer Nature journal content.

In line with the restriction against commercial use, Springer Nature does not permit the creation of a product or service that creates revenue, royalties, rent or income from our content or its inclusion as part of a paid for service or for other commercial gain. Springer Nature journal content cannot be used for inter-library loans and librarians may not upload Springer Nature journal content on a large scale into their, or any other, institutional repository.
These terms of use are reviewed regularly and may be amended at any time. Springer Nature is not obligated to publish any information or content on this website and may remove it or features or functionality at our sole discretion, at any time with or without notice. Springer Nature may revoke this licence to you at any time and remove access to any copies of the Springer Nature journal content which have been saved.
To the fullest extent permitted by law, Springer Nature makes no warranties, representations or guarantees to Users, either express or implied with respect to the Springer nature journal content and all parties disclaim and waive any implied warranties or warranties imposed by law, including merchantability or fitness for any particular purpose.
Please note that these rights do not automatically extend to content, data or other material published by Springer Nature that may be licensed from third parties.
If you would like to use or distribute our Springer Nature journal content to a wider audience or on a regular basis or in any other manner not expressly permitted by these Terms, please contact Springer Nature at

onlineservice@springernature.com